

Appunti didattici

AI, Python, Poetry, RAG, Agents e LangChain

Materiale di studio costruito dagli esercizi svolti durante la conversazione

Argomenti inclusi: Python, Poetry, FastAPI, RAG, Chainlit, OpenAI API, Structured Outputs, Function Calling, Assistants API, Agents, LangChain, LangGraph, Pydantic, ChromaDB, Git/GitHub e integrazione Laravel.

Come usare questi appunti

Leggi prima i capitoli introduttivi per capire il vocabolario, poi passa alle sezioni pratiche con flussi, esempi e comandi. Le checklist finali servono per ripassare prima di consegnare un progetto.

Confronto tra argomenti richiesti e conversazione

Hai chiesto un PDF dettagliato sugli argomenti: Python, Poetry, RAG, Agents e LangChain. Questi argomenti corrispondono al percorso svolto. Durante la conversazione, però, sono emersi anche altri temi fondamentali che vanno aggiunti perché fanno parte dei progetti costruiti.

Argomento	Presente nella conversazione?	Perché è importante
Python	Sì	È il linguaggio usato per tutti i progetti: HR Assistant, FastAPI, Function Calling, Assistants e Travel Agent.
Poetry	Sì	È stato usato per creare ambienti virtuali, gestire dipendenze e avviare comandi.
RAG	Sì	È il cuore del progetto HR Assistant: documenti, chunking, embeddings, ChromaDB, retrieval e risposta LLM.
Agents	Sì	È il tema finale: tool, Agent.run, LangGraph, tracing, modalità test/prod.
LangChain	Sì	È stato usato nei tool del Travel Agent e con LangGraph per collegare funzioni e modello.
FastAPI	Aggiunto	È stato studiato nel selfwork Bookstore API e usato nel Travel Agent API.
Chainlit	Aggiunto	È stato usato come interfaccia chat per HR Assistant e Function Calling.
OpenAI API	Aggiunto	È stata usata per embeddings, structured outputs, function calling e Assistants.
Pydantic	Aggiunto	È stato usato per validare request body, tool input e output strutturati.
ChromaDB	Aggiunto	È il database vettoriale persistente usato nel progetto RAG.
LangGraph	Aggiunto	È stato usato nel progetto finale per creare l'agente ReAct.
Git/GitHub	Aggiunto	È stato usato per commit, push, .gitignore e consegna dei progetti.

Indice degli argomenti

- 1. Python come base dei progetti AI
- 2. Poetry e gestione professionale del progetto
- 3. FastAPI e API backend
- 4. Pydantic e validazione dati
- 5. Git, GitHub e sicurezza dei file
- 6. OpenAI API: embeddings, chat e output strutturati
- 7. RAG: Retrieval Augmented Generation
- 8. HR Assistant: progetto RAG completo
- 9. Chainlit: interfacce chat in Python
- 10. Function Calling e tool use
- 11. Assistants API e file_search
- 12. Agents: concetti fondamentali
- 13. LangChain: catene, prompt e tool
- 14. LangGraph e agenti ReAct
- 15. Travel Agent API: architettura finale
- 16. Debugging, tracing e modalità test/prod
- 17. Errori comuni incontrati e soluzioni
- 18. Checklist di ripasso e consegna

1. Python come base dei progetti AI

Python è stato il linguaggio centrale di tutti i progetti. Lo abbiamo usato per creare API, interfacce chat, logica RAG, tool per agenti e integrazioni con servizi esterni. In un progetto AI moderno, Python non è solo il linguaggio con cui scrivi funzioni: è il collante tra modello LLM, database, API, frontend e strumenti esterni.

Concetti Python usati davvero nei progetti

Concetto	Esempio nel percorso	Perché serve
Moduli e package	src/hr_assistant, src/travel_agent_api	Organizzano il codice in file importabili.
Classi	Config, Database, Agent, ChatLogic	Raccolgono stato e comportamento.
Funzioni	process_documents, run, send_email	Separano compiti piccoli e testabili.
Dizionari e liste	messages, metadata, fake_books_db	Sono il formato principale dei dati scambiati con API e modelli.
Type hints	book_id: int, messages: list	Aiutano FastAPI, Pydantic e l'editor a capire i tipi.
Async/await	Chainlit on_message, chiamate async OpenAI	Permettono di gestire interfacce chat e streaming.
Eccezioni	try/except nei tool SerpApi	Impediscono che un errore esterno blocchi tutta l'app.

Package, moduli e import

Un package Python è una cartella che contiene file Python e di solito un file `__init__.py`. Nel progetto Travel Agent, per esempio, il package è `travel_agent_api`. Questo permette import come:

```
from travel_agent_api.services.agent_service import Agent
from travel_agent_api.tools.hotels_finder import hotels_finder
```

Un errore molto frequente è `ModuleNotFoundError`. Nel percorso è apparso quando Poetry non aveva ancora installato il progetto locale. La soluzione tipica è:

```
poetry install
```

Dizionari e messaggi

Molti sistemi AI usano messaggi in formato lista di dizionari. Per esempio:

```
messages = [
    {"role": "user", "content": "Cerco un hotel 4 stelle a Roma centro"}
]
```

Questa struttura è stata usata nel Travel Agent, nel frontend Laravel e nel router FastAPI. Il modello o l'agente leggono il ruolo e il contenuto per capire chi ha parlato e cosa è stato chiesto.

Regola pratica

Da ricordare

Quando un progetto cresce, non mettere tutto in un unico file. Crea cartelle per routes, services, tools, utils, config. Questo rende il codice più chiaro e più facile da testare.

2. Poetry e gestione professionale del progetto

Poetry è stato usato per creare progetti Python ordinati, gestire dipendenze, ambienti virtuali e comandi di avvio. È più strutturato rispetto a un semplice requirements.txt perché usa pyproject.toml e poetry.lock.

Comandi principali

Comando	Cosa fa
poetry new nome-progetto	Crea un nuovo progetto con struttura standard.
poetry add fastapi uvicorn	Aggiunge dipendenze al progetto.
poetry install	Installa dipendenze e package locale.
poetry run python file.py	Esegue Python dentro l'ambiente Poetry.
poetry run uvicorn modulo:app --reload	Avvia una API FastAPI.
poetry env use python3.12	Forza Poetry a usare una versione specifica di Python.
poetry env info	Mostra informazioni sull'ambiente virtuale.
poetry check	Controlla pyproject.toml.

pyproject.toml

pyproject.toml è il file principale del progetto. Contiene nome, versione, dipendenze e vincoli su Python. Esempio:

```
[project]
name = "travel-agent-api"
version = "0.1.0"
requires-python = ">=3.12,<4.0.0"
```

Durante gli esercizi abbiamo visto che il vincolo requires-python è importante. Chainlit, per esempio, non era compatibile con Python 3.14 o superiore, quindi bisognava impostare un limite come `>=3.12,<3.14.0`.

poetry.lock

poetry.lock blocca le versioni esatte delle librerie installate. Questo rende il progetto riproducibile: chi clona il repository può ottenere lo stesso ambiente con poetry install.

Poetry vs requirements.txt

Uso	File consigliato
Progetto con Poetry	pyproject.toml + poetry.lock
Progetto semplice con pip	requirements.txt
Consegna GitHub con Poetry	Non serve requirements.txt, salvo richiesta esplicita.

Errore comune

Se il comando `import nome_package` fallisce, controlla di essere nella cartella giusta e lancia `poetry install`. Questo installa anche il package locale del progetto.

3. FastAPI e API backend

FastAPI è stato studiato nel selfwork Bookstore API e poi riutilizzato nel progetto finale Travel Agent. Serve per creare API HTTP in Python. È molto adatto ai progetti AI perché espone facilmente endpoint che possono essere chiamati da frontend, app web o altri servizi.

Struttura base

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def root():
    return {"message": "API attiva"}
```

Uvicorn

Uvicorn è il server ASGI usato per avviare FastAPI. Nel progetto Travel Agent abbiamo usato la porta 8080 perché Laravel usa 8000.

```
poetry run uvicorn travel_agent_api.main:app --reload --port 8080
```

Path parameters

I path parameters sono parti dinamiche dell'URL. Esempio:

```
@app.get("/books/{book_id}")
def get_book(book_id: int):
    return {"book_id": book_id}
```

Se visiti /books/1, FastAPI converte 1 in int. Se visiti /books/ciao, FastAPI restituisce un errore di validazione.

Query parameters

I query parameters stanno dopo il punto interrogativo nell'URL. Esempio: /books/?limit=2&available=true

```
@app.get("/books/")
def get_books(limit: int = 10, available: bool | None = None):
    return {"limit": limit, "available": available}
```

Request body

Il request body è il JSON inviato con POST o PUT. FastAPI usa Pydantic per validarlo.

```
from pydantic import BaseModel

class Book(BaseModel):
    title: str
    price: float

@app.post("/books/")
def create_book(book: Book):
    return book
```

Router e CORS

Nel Travel Agent abbiamo separato le rotte in `routes/chat_router.py`. `main.py` include il router con prefix `/chat`. Abbiamo anche configurato CORS per permettere al frontend Laravel su `localhost:8000` di chiamare l'API Python su `127.0.0.1:8080`.

```
app.include_router(chat_router.router, tags=["Chat"], prefix="/chat")
```

Da ricordare

FastAPI genera automaticamente la documentazione su `/docs` e `/redoc`. Questo è utilissimo per testare API prima di collegare il frontend.

4. Pydantic e validazione dati

Pydantic è stato usato in FastAPI, Structured Outputs e LangChain tools. Serve per dichiarare la forma dei dati e validare automaticamente input e output.

BaseModel

```
from pydantic import BaseModel, Field

class HotelInput(BaseModel):
    q: str
    hotel_class: int = Field(3, ge=2, le=5)
```

In questo esempio `hotel_class` deve essere almeno 2 e massimo 5. Se arriva 6, Pydantic genera errore. Lo abbiamo visto nel test del tool `hotels_finder`.

Field

Field permette di aggiungere descrizioni e vincoli. Nei tool LangChain le descrizioni sono importanti perché aiutano il modello a capire quando e come chiamare un tool.

model_dump

`model_dump` converte un oggetto Pydantic in dizionario Python. È utile quando vogliamo salvare o restituire dati.

```
book_dict = book.model_dump()
```

Annotated, Query e Path

```
from typing import Annotated
from fastapi import Query, Path

def route(limit: Annotated[int, Query(ge=1, le=20)] = 10):
    ...
```

Questa sintassi rende espliciti i vincoli e li mostra anche nella documentazione Swagger.

5. Git, GitHub e sicurezza dei file

Durante gli esercizi abbiamo usato Git per salvare ogni fase con commit semplici in italiano. Questo rende il percorso leggibile anche a chi controlla il progetto.

Commit semplici

```
git add .
git commit -m "Aggiunto tool ricerca hotel"
git push
```

.gitignore

Il file .gitignore evita di caricare file sensibili o inutili su GitHub.

```
.env
.venv/
__pycache__/
*.pyc
.DS_Store
.vscode/
```

.env e .env.example

File	Va su GitHub?	Contenuto
.env	No	Chiavi vere e configurazioni locali.
.env.example	Sì	Template senza segreti.

```
OPENAI_API_KEY=""
SERPAPI_API_KEY=""
API_MODE="test"
MODEL_NAME="gpt-4o"
```

Regola di sicurezza

Non fare mai commit del file .env. Se una chiave viene caricata per errore, va considerata compromessa e va rigenerata.

6. OpenAI API: embeddings, chat e output strutturati

OpenAI API è stata usata in diversi modi: embeddings per RAG, chat completion per risposte, Structured Outputs per JSON validato, Function Calling per chiamare funzioni e Assistants API per agenti con `file_search`.

Embeddings

Un embedding è una rappresentazione numerica di un testo. Testi semanticamente simili hanno embeddings vicini nello spazio vettoriale. Nel progetto HR Assistant abbiamo usato `text-embedding-3-small` per indicizzare CV e documenti.

Structured Outputs

Structured Outputs significa chiedere al modello di rispondere in una struttura precisa, per esempio un modello Pydantic. Questo rende la risposta utilizzabile da Python senza dover interpretare testo libero.

```
class SupportTicketAnalysis(BaseModel):
    summary: str
    category: str
    priority: str
    needs_human_operator: bool
```

Function Calling

Function Calling permette al modello di scegliere una funzione e generare gli argomenti corretti. Python esegue la funzione e rimanda il risultato al modello.

Fase	Cosa succede
1	L'utente chiede qualcosa.
2	Il modello decide se serve una funzione.
3	Il modello produce nome funzione e argomenti.
4	Python esegue la funzione.
5	Il risultato torna al modello.
6	Il modello risponde all'utente.

Assistants API

Assistants API introduce concetti come `assistant`, `thread`, `run`, `required_action` e `tool_outputs`. Nel progetto TechFix l'assistente usava `file_search` e una funzione `send_email`.

7. RAG: Retrieval Augmented Generation

RAG significa Retrieval Augmented Generation. È una tecnica in cui il modello non risponde solo con la sua conoscenza interna, ma prima recupera informazioni da documenti esterni e poi genera una risposta basata su quei contenuti.

Flusso RAG

Passaggio	Descrizione
1. Caricamento documenti	PDF, TXT, DOCX, CSV, JSON, ZIP o altri formati.
2. Estrazione testo	Il documento viene convertito in testo leggibile.
3. Chunking	Il testo viene diviso in pezzi più piccoli.
4. Embeddings	Ogni chunk diventa un vettore numerico.
5. Vector database	I vettori vengono salvati in ChromaDB.
6. Query utente	Anche la domanda viene trasformata in embedding.
7. Retrieval	Si recuperano i chunk più simili alla domanda.
8. Generation	Il modello genera la risposta usando i chunk recuperati.

Perché serve il RAG

- Permette al modello di usare documenti privati o aggiornati.
- Riduce le allucinazioni perché la risposta è ancorata a fonti recuperate.
- Evita di inserire documenti lunghissimi direttamente nel prompt.
- Permette ricerca semantica, non solo per parola chiave.

Chunking

Il chunking è la divisione del testo in parti. Nel progetto HR Assistant abbiamo usato strategie fixed, paragraph, section e semantic. Il chunking è importante perché chunk troppo grandi sono difficili da recuperare con precisione, mentre chunk troppo piccoli perdono contesto.

Metadata

I metadata descrivono ogni chunk: nome file, hash, ultima modifica, tipo file, nome candidato, email, telefono. Servono per filtrare, evitare duplicati e ricostruire la provenienza della risposta.

ChromaDB

ChromaDB è il vector database usato per salvare embeddings e documenti. Nel progetto HR Assistant è stato configurato con PersistentClient in data/chromadb, così l'indice rimane salvato tra un riavvio e l'altro.

```
client = chromadb.PersistentClient(path="data/chromadb")
```

8. HR Assistant: progetto RAG completo

HR Assistant è stato il progetto RAG più completo. L'obiettivo era cercare candidati e informazioni dentro CV e documenti indicizzati.

Componenti principali

File	Responsabilità
config.py	Configurazione modelli, directory, ChromaDB, chunking.
document_processor.py	Lettura documenti, estrazione testo, sincronizzazione.
database.py	Gestione ChromaDB, query, add, remove, count.
chunking.py	Strategie di chunking.
utils.py	Chiamate LLM, estrazione candidato, streaming.
__init__.py	Interfaccia Chainlit.

Sincronizzazione incrementale

Il sistema non reindicizza sempre tutto. Usa hash e metadata per capire se un documento è nuovo, modificato, eliminato o invariato. Questo rende l'indicizzazione più efficiente.

Formati supportati

Con MarkItDown abbiamo aggiunto supporto a diversi formati: TXT, PDF, DOC, DOCX, CSV, JSON, XLSX, PPTX, HTML, XML e ZIP.

Upload da interfaccia

Con Chainlit abbiamo aggiunto il pulsante per caricare documenti direttamente dall'interfaccia. I file vengono salvati nella cartella resumes e poi processati.

Lezione importante

RAG non è solo “mettere documenti nel modello”

Un buon RAG richiede ingestion, pulizia testo, chunking, embeddings, metadata, retrieval, prompt e interfaccia. Ogni pezzo influenza la qualità della risposta.

9. Chainlit: interfacce chat in Python

Chainlit è stato usato per creare interfacce chat in modo rapido. È utile quando vuoi testare un assistente AI senza costruire un frontend completo.

Elementi principali

```
import chainlit as cl

@cl.on_chat_start
async def start():
    await cl.Message(content="Ciao").send()

@cl.on_message
async def main(message: cl.Message):
    await cl.Message(content="Risposta").send()
```

Azioni e pulsanti

Nel progetto HR Assistant abbiamo usato azioni per sincronizzare CV, vedere statistiche, caricare documenti e resettare l'indice.

Streaming

Lo streaming permette di mostrare la risposta del modello mentre viene generata. È utile per migliorare l'esperienza utente.

Personalizzazione grafica

Abbiamo personalizzato Chainlit con avatars, theme.json e chainlit.md, rendendo l'interfaccia più adatta al progetto.

10. Function Calling e tool use

Il Function Calling è stato studiato con un esercizio ristorante. L'obiettivo era far scegliere al modello funzioni come menu, disponibilità tavoli e prenotazione.

Differenza tra risposta normale e function calling

Risposta normale	Function Calling
Il modello risponde con testo.	Il modello può chiedere di eseguire una funzione.
Non agisce sul mondo esterno.	Python esegue logica reale o simulata.
Meno controllabile.	Più strutturato e testabile.

Schema tipico

```
TOOLS = [  
  {  
    "type": "function",  
    "function": {  
      "name": "book_table",  
      "description": "Prenota un tavolo",  
      "parameters": {...}  
    }  
  }  
]
```

Esercizio ristorante

- MenuTool: informazioni sul menu e preferenze alimentari.
- AvailabilityTool: verifica disponibilità tavoli.
- BookingTool: prenota un tavolo.

Questo esercizio ha preparato il terreno per gli agenti, perché un agente è in pratica un modello che sa decidere quali strumenti usare in base alla richiesta.

11. Assistants API e file_search

Nel progetto TechFix Assistant abbiamo usato OpenAI Assistants con `file_search` e una funzione `send_email`. Questo progetto mostrava un altro modo di costruire assistenti: invece di gestire tutto manualmente, si crea un assistant con tool collegati.

Concetti principali

Concetto	Significato
Assistant	Configurazione stabile del comportamento e dei tool.
Thread	Conversazione con l'utente.
Run	Esecuzione dell'assistant su un thread.
<code>file_search</code>	Tool per cercare nei file caricati in un vector store.
<code>requires_action</code>	Stato in cui l'assistant chiede a Python di eseguire una funzione.
<code>submit_tool_outputs</code>	Invio del risultato funzione all'assistant.

TechFix Assistant

Il progetto simulava un centro assistenza tecnica. L'assistente rispondeva su riparazioni, prezzi, orari e garanzia usando una knowledge base. Se l'utente forniva nome ed email, poteva attivare la funzione `send_email`.

Modalità simulazione email

Abbiamo gestito il caso in cui `RESEND_KEY` non fosse configurata. In quel caso il sistema non invia una vera email, ma restituisce un messaggio di simulazione.

12. Agents: concetti fondamentali

Un agent è un sistema in cui un modello linguistico non si limita a rispondere, ma può decidere quali strumenti usare, con quali argomenti e in quale ordine. Gli agents sono il tema centrale del progetto finale Agentic AI & Python.

Differenza tra chatbot e agent

Chatbot semplice	Agent
Risponde al messaggio.	Decide se usare tool.
Non ha azioni strutturate.	Può chiamare funzioni e API.
Ha meno controllo sul processo.	Può seguire un ciclo ragionamento-azione-osservazione.
Adatto a FAQ semplici.	Adatto a compiti multi-step.

Componenti di un agent

- Modello LLM: interpreta la richiesta.
- System prompt: definisce ruolo e regole.
- Tools: funzioni disponibili.
- Tool schema: descrive input e output del tool.
- Executor: gestisce il ciclo agentico.
- Tracing: mostra cosa viene eseguito.

ReAct

ReAct significa Reasoning + Acting. L'agente ragiona su cosa fare, sceglie un'azione/tool, osserva il risultato e continua fino alla risposta finale. Nel Travel Agent abbiamo usato `create_react_agent` di LangGraph in modalità produzione.

Modalità test e prod

Nel progetto finale abbiamo aggiunto una modalità test per non chiamare OpenAI e SerpApi durante lo sviluppo. Questa è una personalizzazione utile e sicura.

```
API_MODE="test" # risposte simulate
API_MODE="prod" # OpenAI + LangGraph + SerpApi
```

13. LangChain: catene, prompt e tool

LangChain è un framework per costruire applicazioni basate su LLM. Nel percorso lo abbiamo usato per prompt, tool, output parser e integrazione OpenAI.

ChatPromptTemplate

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "Sei un esperto di viaggi"),
    ("human", "{input}")
])
```

Pipe operator

LangChain permette di collegare componenti con il simbolo `|`. Esempio: `prompt | model | parser`.

```
chain = prompt | model | output_parser
result = chain.invoke({"input": "Crea un itinerario"})
```

@tool

Il decoratore `@tool` trasforma una funzione Python in uno strumento che un agent può usare.

```
from langchain_core.tools import tool

@tool
def chain_historical_expert(input_text: str) -> str:
    return "Risposta storica"
```

PydanticOutputParser

PydanticOutputParser forza l'output del modello a rispettare un modello Pydantic. È utile quando vuoi risultati strutturati, per esempio un piano viaggio diviso in giornate.

Tool schema

Nei tool voli e hotel abbiamo definito classi input Pydantic. Queste classi dicono all'agente quali parametri servono e che tipo devono avere.

14. LangGraph e agenti ReAct

LangGraph è stato usato nel progetto finale per creare un agent ReAct tramite `create_react_agent`. Rispetto a una chain lineare, LangGraph è pensato per flussi agentici con più passaggi e tool.

create_react_agent

```
from langgraph.prebuilt import create_react_agent

self.tools = [
    chain_historical_expert,
    flights_finder,
    hotels_finder,
    chain_travel_plan,
]

self.agent_executor = create_react_agent(self.model, self.tools)
```

Come lavora l'Agent nel Travel Agent

Fase	Descrizione
Input	Riceve lista messages da Laravel o Swagger.
System prompt	Aggiunge ruolo: sei un travel planner.
Tool list	Mette a disposizione storico, voli, hotel, piano viaggio.
Agent executor	Invoca LangGraph ReAct agent.
Output	Restituisce la lista dei messaggi al client.

Perché abbiamo aggiunto run_test_mode

`run_test_mode` permette di verificare il flusso senza chiamare servizi esterni. In base alle parole della richiesta, riconosce hotel, voli, storia o itinerario e risponde con un messaggio simulato.

15. Travel Agent API: architettura finale

Travel Agent API è il progetto finale della specializzazione Agentic AI & Python. È una API Python con FastAPI, LangChain, LangGraph, OpenAI e SerpApi. Si integra con un frontend Laravel.

Struttura finale

```
travel-agent-api/
├── README.md
├── pyproject.toml
├── poetry.lock
├── .env.example
├── src/
│   └── travel_agent_api/
│       ├── config.py
│       ├── main.py
│       ├── routes/chat_router.py
│       ├── services/agent_service.py
│       └── tools/
│           ├── chain_historical_expert.py
│           ├── chain_travel_plan.py
│           ├── flights_finder.py
│           └── hotels_finder.py
```

Flusso completo

Passaggio	File coinvolto
Laravel invia messaggio	Chatbot.php
FastAPI riceve POST /chat/travel-agent	chat_router.py
Router crea Agent	chat_router.py
Agent decide modalità test/prod	agent_service.py
Agent usa tool se necessario	tools/*.py
Risposta torna al frontend	chat_router.py

Tool del Travel Agent

Tool	Scopo	Modalità test
chain_historical_expert	Informazioni storiche e culturali	Risposta storica simulata.
chain_travel_plan	Crea itinerario	Piano viaggio demo.
flights_finder	Cerca voli	Volo demo.
hotels_finder	Cerca hotel	Hotel demo.

CORS e Laravel

Il frontend Laravel gira su localhost:8000. L'API Python gira su 127.0.0.1:8080. Senza CORS, il browser potrebbe bloccare le chiamate. Per questo main.py autorizza <http://127.0.0.1:8000> e <http://localhost:8000>.

16. Debugging, tracing e modalità test/prod

La traccia del progetto finale chiedeva di inserire print per fare tracing dell'esecuzione. Il tracing serve a capire cosa accade quando l'utente invia un messaggio.

Esempio tracing

```
print("'" * 80)
print("Agent.run")
print("'" * 80)
print("api_mode:", self.api_mode)
print("messages:", messages)
```

Dove abbiamo aggiunto tracing

- chat_completion: quando arriva una richiesta HTTP.
- Agent.__init__: quando viene creato l'agente.
- Agent.run: quando l'agente elabora i messaggi.
- chain_historical_expert: quando parte il tool storico.
- chain_travel_plan: quando parte il tool itinerario.
- flights_finder: quando parte il tool voli.
- hotels_finder: quando parte il tool hotel.

Modalità test

In test non vengono chiamate API esterne. Questo protegge da costi, errori di chiavi e dipendenze esterne durante lo sviluppo.

Modalità prod

In prod il progetto usa OpenAI, LangGraph e SerpApi. Serve configurare OPENAI_API_KEY e SERPAPI_API_KEY.

17. Errori comuni incontrati e soluzioni

Errore	Causa probabile	Soluzione
ModuleNotFoundError: no module named ...	Package locale non installato o import errato.	poetry install e controllo percorso import.
Uvicorn: Attribute app not found	Nel file non esiste app = FastAPI() o comando sbagliato.	Controllare main.py e usare modulo:app corretto.
Python version not compatible	requires-python troppo largo o libreria non compatibile.	Impostare versione corretta in pyproject.toml.
CORS error	Frontend e backend su porte diverse.	Configurare CORSMiddleware.
Pydantic ValidationError	Input fuori dai vincoli.	Controllare Field, Query, Path e body JSON.
Composer install fallisce con PHP 8.5	Dipendenze Laravel compatibili solo fino a PHP 8.4.	Usare PHP 8.4 e composer install.
.env su GitHub	File sensibile tracciato.	Aggiungere .env a .gitignore e rimuoverlo dal tracking.

Metodo generale di debug

1. Leggi l'ultima riga dell'errore.
2. Capisci se è un problema di import, versione, configurazione o input.
3. Testa un pezzo piccolo alla volta.
4. Aggiungi print di tracing.
5. Rilancia il test diretto prima dell'interfaccia completa.

18. Checklist di ripasso e consegna

Checklist progetto Python

- pyproject.toml e poetry.lock presenti.
- poetry install funziona.
- README.md completo.
- .env non tracciato.
- .env.example presente.
- API avviabile con uvicorn.
- /docs funzionante.
- /health funzionante.
- Endpoint principale testato.
- Tracing visibile nel terminale.
- Commit puliti e comprensibili.

Checklist RAG

- Documenti caricati e convertiti in testo.
- Chunking scelto e motivato.
- Embeddings generati.
- Vector database persistente.
- Metadata utili.
- Retrieval testato.
- Risposte basate sui documenti.

Checklist Agents

- Tool definiti con descrizioni chiare.
- Input dei tool validati con Pydantic.
- Agent collegato ai tool.
- System prompt chiaro.
- Modalità test disponibile.
- Tracing per capire quale tool parte.
- README spiega personalizzazioni.

Ripasso rapido: differenze importanti

Concetto	Definizione breve
RAG	Recupera documenti rilevanti e poi genera una risposta.
Function Calling	Il modello chiede a Python di eseguire una funzione.
Agent	Sistema che sceglie strumenti e passaggi per completare un compito.
LangChain	Framework per prompt, chain, tool e parser.
LangGraph	Framework per agenti e workflow più strutturati.
Pydantic	Validazione e struttura dei dati.
FastAPI	API HTTP rapide e documentate automaticamente.
Poetry	Gestione progetto, dipendenze e ambiente virtuale.

Conclusione

Il percorso svolto copre una pipeline completa di sviluppo AI in Python: dal progetto base con Poetry fino a RAG, Function Calling, Assistants, FastAPI e Agents con LangChain/LangGraph. La competenza più importante non è solo conoscere le librerie, ma sapere come collegarle in architetture pulite, testabili e sicure.

Per studiare, concentrati sui flussi: input utente, validazione, retrieval o tool, chiamata modello, risposta finale. Quasi tutti i progetti AI seguono una variante di questo schema.

Appendice didattica: lezioni operative dettagliate

Questa appendice trasforma gli argomenti in materiale di studio più pratico. Ogni sezione spiega non solo cosa significa un concetto, ma anche come lo hai usato nei progetti, quando usarlo e quali errori evitare.

A. Come leggere un progetto Python AI

Quando apri un progetto AI in Python, non partire subito dal codice più difficile. Parti dalla struttura. La struttura ti dice già quali responsabilità ha ogni parte del sistema.

Cartella o file	Cosa controllare	Domanda guida
pyproject.toml	Dipendenze e versione Python	Con quale ambiente devo eseguire il progetto?
.env.example	Variabili richieste	Quali chiavi o configurazioni servono?
main.py	Punto di ingresso API o app	Come parte il server?
routes/	Endpoint disponibili	Quali URL espone il backend?
services/	Logica principale	Dove vive il cervello dell'app?
tools/	Funzioni chiamabili da agenti	Quali azioni può fare il modello?
README.md	Istruzioni e documentazione	Come installo, avvio e testo?

Questa lettura è stata usata nel progetto Travel Agent: prima abbiamo capito il client Laravel, poi abbiamo creato un backend Python compatibile con il suo endpoint.

B. Python: pattern di architettura usati

Config object

Un oggetto di configurazione centralizza valori usati in più parti del progetto. Nel Travel Agent abbiamo creato settings in config.py.

```
class Settings:
    API_MODE = os.getenv("API_MODE", "test").lower()
    OPENAI_API_KEY = os.getenv("OPENAI_API_KEY", "")
    SERPAPI_API_KEY = os.getenv("SERPAPI_API_KEY", "")
    MODEL_NAME = os.getenv("MODEL_NAME", "gpt-4o")

settings = Settings()
```

Vantaggio: se vuoi cambiare modello o modalità, modifichi .env e non il codice sparso in tanti file.

Service object

Un service contiene logica di business. Nel Travel Agent, Agent è un service perché coordina modello, tool e messaggi.

```
class Agent:
    def __init__(self):
        self.tools = [flights_finder, hotels_finder]

    def run(self, messages: list):
        return self.run_test_mode(messages)
```

Tool function

Un tool è una funzione specializzata. Deve avere uno scopo chiaro, input validato e output prevedibile.

```
@tool(args_schema=HotelsInputSchema)
def hotels_finder(params: HotelsInput) -> dict | list | str:
    ...
```

C. Poetry: workflow completo da usare sempre

Questo è il flusso consigliato quando inizi un progetto Python professionale.

6. Crea il progetto con `poetry new nome-progetto`.
7. Entra nella cartella del progetto.
8. Imposta la versione Python corretta con `poetry env use python3.12`.
9. Controlla `pyproject.toml` e `requires-python`.
10. Aggiungi dipendenze con `poetry add`.
11. Installa il package locale con `poetry install`.
12. Esegui test minimi di import.
13. Crea `.gitignore` e commit iniziale.

```
poetry new travel-agent-api
cd travel-agent-api
poetry env use python3.12
poetry add fastapi uvicorn pydantic
poetry install
poetry run python -c "import fastapi; print('ok')"
```

Perché poetry install è importante

`poetry add` installa le dipendenze. `poetry install` installa anche il progetto locale, permettendo import come `import travel_agent_api`.

D. FastAPI: lifecycle di una richiesta

Una richiesta HTTP in FastAPI attraversa più passaggi. Capire questo flusso aiuta a debuggare.

Fase	Cosa succede	Dove lo vedi nel codice
1. Richiesta arriva	Browser, Swagger o Laravel invia HTTP request	POST /chat/travel-agent
2. Router selezionato	FastAPI cerca la rotta corrispondente	chat_router.py
3. Validazione body	Pydantic controlla il JSON	ChatCompletionRequest
4. Logica business	La funzione chiama un service	Agent.run
5. Risposta	FastAPI converte dict/list in JSON	return response

```
@router.post("/travel-agent")
def chat_completion(request: ChatCompletionRequest):
    agent = Agent()
    response = agent.run(messages=request.messages)
    return response
```

Nel progetto Travel Agent il router non contiene la logica dell'agente. Il router riceve e inoltra. La logica sta in services/agent_service.py. Questa separazione è un buon refactoring.

E. RAG spiegato come pipeline tecnica

RAG è una pipeline. Se una risposta RAG non è buona, devi capire in quale fase si rompe.

Problema osservato	Possibile fase difettosa	Cosa controllare
Il documento non viene trovato	Ingestion	File letto correttamente? Formato supportato?
Risposta poco precisa	Chunking	Chunk troppo grandi o troppo piccoli?
Risultati non pertinenti	Embeddings/retrieval	Query simile al contenuto? top_k adeguato?
Risposta inventata	Prompt generation	Il modello è obbligato a usare solo il contesto?
Dati vecchi	Sync metadata	Hash e last_modified aggiornati?

Ingestion

L'ingestion è la fase di lettura dei documenti. Nel progetto HR Assistant abbiamo usato MarkItDown per convertire tanti formati in testo.

Embeddings

Gli embeddings non sono riassunti: sono coordinate semantiche. Servono a confrontare similarità tra domanda e documenti.

Retrieval

Il retrieval recupera i chunk più simili alla domanda. La qualità dipende da embeddings, chunking e metadata.

Generation

Il modello riceve domanda + contesto recuperato e produce una risposta. Il prompt deve dire chiaramente di non inventare se il contesto non contiene la risposta.

F. RAG vs file_search degli Assistants

RAG manuale e file_search sono collegati, ma non sono identici.

Aspetto	RAG manuale	Assistants file_search
Controllo chunking	Alto	Gestito principalmente da OpenAI
Vector database	Scegli tu, es. ChromaDB	Vector store OpenAI
Codice retrieval	Lo scrivi tu	Gestito dal tool file_search
Personalizzazione	Molto alta	Più semplice ma meno controllabile
Caso d'uso	Sistemi custom complessi	Assistenti rapidi con file allegati

HR Assistant usa un RAG manuale con ChromaDB. TechFix Assistant usa Assistants API con file_search. Entrambi recuperano informazioni da documenti, ma con livelli diversi di controllo.

G. Structured Outputs: perché sono diversi da JSON libero

Chiedere “rispondi in JSON” non basta. Il modello potrebbe sbagliare campi, tipi o formato. Structured Outputs usa uno schema preciso, spesso Pydantic, e rende la risposta più affidabile.

```
class RealEstateSearchParameters(BaseModel):  
    city: str  
    min_price: int | None = None  
    max_price: int | None = None  
    rooms: int | None = None
```

Quando usare Structured Outputs:

- estrarre dati da testo libero;
- classificare ticket;
- trasformare una richiesta utente in parametri di ricerca;
- generare dati che poi Python deve usare.

Quando non basta: se devi eseguire azioni reali, serve Function Calling o un agent.

H. Function Calling: schema mentale definitivo

Function Calling non significa che il modello esegue codice. Il modello decide quale funzione chiamare e con quali argomenti. Python esegue davvero la funzione.

Responsabilità	Modello	Python
Capire richiesta	Sì	No
Scegliere funzione	Sì	No
Creare argomenti	Sì	Validazione
Eseguire funzione	No	Sì
Accedere a database/API	No	Sì
Formulare risposta finale	Sì	Può preparare dati

Nel ristorante, il modello poteva scegliere `get_menu_info`, `get_available_tables` o `book_table`. Nel Travel Agent, un agent può scegliere voli, hotel, itinerario o storia.

I. Assistants API: thread e run in pratica

Assistants API organizza la conversazione in thread e run. Questo è diverso da una semplice chiamata chat.

Elemento	Spiegazione didattica
Assistant	Configurazione stabile: istruzioni, modello e tool.
Thread	Contenitore della conversazione con un utente.
Message	Singolo messaggio dentro un thread.
Run	Esecuzione dell'assistant sui messaggi del thread.
requires_action	L'assistant chiede di eseguire una funzione.
tool_outputs	Risultati delle funzioni eseguite da Python.

Nel progetto TechFix, `create_assistant.py` creava assistant e vector store. `chat_logic.py` gestiva thread, run e function call `send_email`.

J. Agents: come progettare un tool buono

Un agent è efficace solo se i suoi tool sono progettati bene. Un tool confuso porta l'agente a usarlo male.

Regola	Esempio pratico
Nome chiaro	hotels_finder comunica subito lo scopo.
Descrizione chiara	Dice quando usarlo e cosa restituisce.
Input validato	HotelsInput controlla q, date, adulti, hotel_class.
Output prevedibile	In test restituisce dict con mode, message, hotels.
Gestione errori	Se manca SERPAPI_API_KEY restituisce messaggio chiaro.
Tracing	Stampa il nome del tool quando parte.

```
if not serpapi_key:  
    return "SERPAPI_API_KEY non configurata nel file .env."
```

K. LangChain: quando usare chain e quando usare agent

Situazione	Soluzione consigliata
Un solo passaggio prevedibile	Chain
Output strutturato da un prompt	Chain + PydanticOutputParser
Scelta tra più strumenti	Agent
Interazione con API esterne	Agent con tools
Workflow con stati complessi	LangGraph

chain_travel_plan è una chain: input viaggio -> prompt -> modello -> parser. Travel Agent è un agent: l'utente può chiedere hotel, voli, storia o itinerario, quindi il sistema deve scegliere il tool.

L. Materiale pratico: comandi finali da conoscere

Obiettivo	Comando
Installare progetto	poetry install
Aggiungere libreria	poetry add nome-libreria
Controllare import	poetry run python -c "import fastapi"
Avviare FastAPI Travel Agent	poetry run uvicorn travel_agent_api.main:app --reload --port 8080
Avviare FastAPI semplice	poetry run uvicorn main:app --reload
Controllare sintassi	poetry run python -m py_compile file.py
Vedere stato Git	git status
Commit	git add . && git commit -m "Messaggio"
Push	git push

M. Domande di ripasso

- Che differenza c'è tra path parameter e query parameter?
- Perché .env non va caricato su GitHub?
- A cosa serve poetry.lock?
- Qual è la differenza tra RAG e fine-tuning?
- Perché un RAG ha bisogno di chunking?
- Cosa fa un embedding?
- Che differenza c'è tra Function Calling e Agent?
- Cosa fa @tool in LangChain?
- Perché Pydantic è utile nei tool?
- Cosa cambia tra API_MODE test e prod?
- A cosa serve CORS nel Travel Agent?
- Perché LangGraph è utile per agenti?

N. Mini glossario

Termine	Definizione da ricordare
API	Interfaccia che permette a programmi diversi di comunicare.
Endpoint	URL specifico di una API.
JSON	Formato dati usato per richieste e risposte web.
LLM	Modello linguistico di grandi dimensioni.
Embedding	Vettore numerico che rappresenta il significato di un testo.
Vector DB	Database ottimizzato per cercare vettori simili.
Chunk	Pezzo di testo ricavato da un documento.
Tool	Funzione che un modello o agent può usare.
Agent	Sistema AI che sceglie azioni e strumenti.
Tracing	Stampe/log che mostrano cosa viene eseguito.
CORS	Regola browser che controlla chiamate tra domini/porte diverse.

O. Schema mentale finale

Puoi leggere tutti i progetti della conversazione con questo schema unico:

Domanda	Risposta nel progetto
Chi manda l'input?	Utente, Chainlit, Swagger, Laravel.
Chi riceve?	FastAPI route o Chainlit callback.
Chi valida?	Pydantic o codice Python.
Chi decide?	Funzione, Assistant o Agent.
Chi agisce?	Tool, database, API esterna.
Chi genera testo?	LLM OpenAI/Ollama.
Chi mostra la risposta?	Chainlit, Swagger, Laravel.

Questa è la logica più importante da portare avanti nei prossimi progetti AI.